

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Edytor schematów szydełkowych z automatycznym rozmieszczaniem symboli

Etap 2: Projekt architektury i projekt szczegółowy systemu

Autorzy:

Anna Babicka, Wiktor Murawski

Promotor:

dr inż. Paweł Kotowski

29 października 2026

Spis treści

1 Architektura systemu **4**
1.1 Przegląd logiczny 4
1.2 Odpowiedzialności modułów 4

2 Projekt modułów **5**
2.1 Warstwy i zależności 5
2.2 Moduły szczegółowe (katalogi) 6

3 Komunikacja **8**

4 Główne komponenty – opisy i diagramy **9**
4.1 Model danych (Domain) 9
4.2 Stany aplikacji (uproszczony diagram stanów) 9
4.3 Aktywność: dodanie oczka (US-02) 9
4.4 Zdarzenia kluczowe 9

5 Wizja interfejsu użytkownika **11**
5.1 Główne obszary (wireframe tekstowy) 11
5.2 Kluczowe interakcje 11

6 Zewnętrzne interfejsy **12**
6.1 Format pliku projektu (JSON) 12
6.2 Eksport graficzny 12

7 Stos technologiczny **13**
7.1 Uzasadnienie wyboru podejścia do layoutu 14
7.2 Środowisko deweloperskie 14

Historia zmian dokumentu

Wersja	Data	Autor	Opis zmian
1.0	29.10.2026	Wiktor Murawski	*Pierwsza wersja dokumentu – szkielet architektury, wybór technologii, opis modułów*

1 Architektura systemu

1.1 Przegląd logiczny

Diagram przedstawia ogólną, logiczną architekturę systemu: główne moduły aplikacji i zależności między nimi, bez odwołań do konkretnych technologii. Strzałki wskazują kierunek wywołań między modułami; moduł wywoływany zwraca wynik do modułu wywołującego. Linia przerywana oznacza dostęp tylko do odczytu.

System składa się z trzech głównych części.

Moduł edytora odpowiada za interakcję z użytkownikiem. Warstwa UI umożliwia edycję i wyświetla schemat, a kontroler koordynuje przepływ informacji między interfejsem a pozostałymi modułami. Kontroler przechowuje także bieżący stan aplikacji i historię operacji, dzięki czemu warstwa UI jest odizolowana od logiki dziedzinowej.

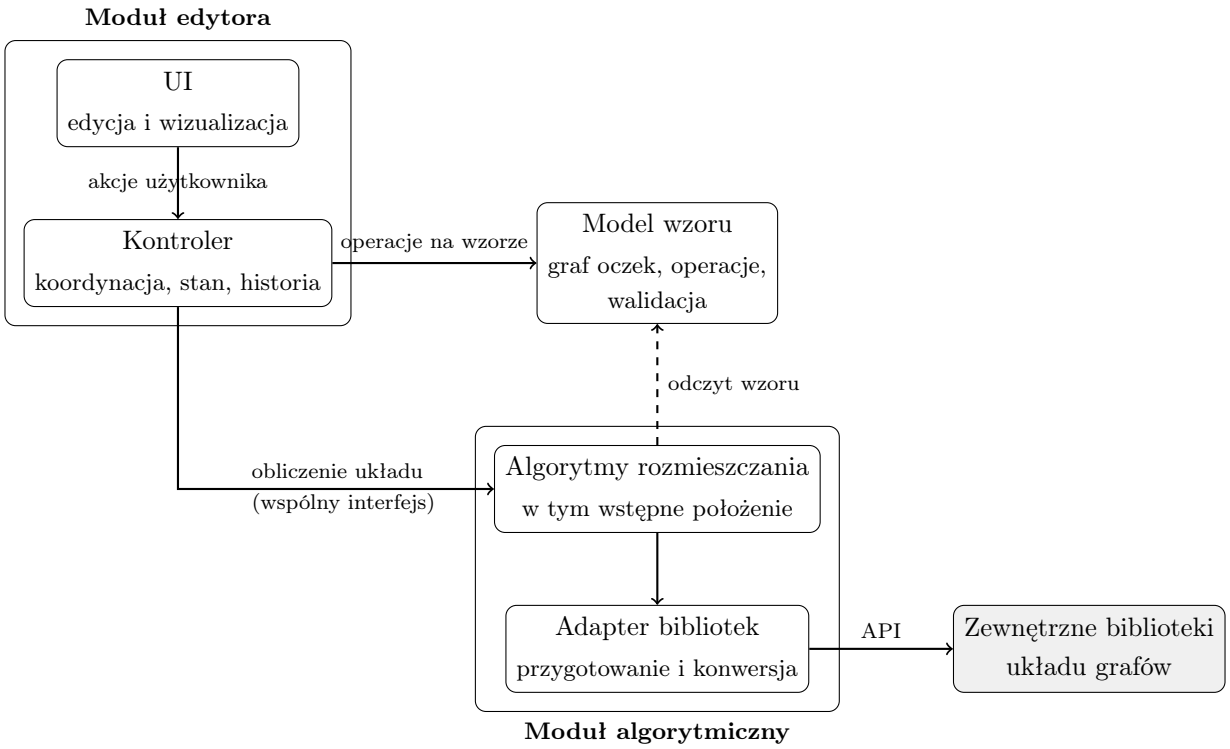
Model wzoru przechowuje formalną reprezentację wzoru w postaci grafu oczek wraz z operacjami jego modyfikacji i regułami zapewniającymi, że wzór jest możliwy do wykonania. Stanowi jedyne źródło danych o strukturze wzoru. Zmiany wprowadza wyłącznie kontroler, a moduł algorytmiczny jedynie odczytuje strukturę wzoru.

Moduł algorytmiczny na podstawie struktury wzoru wyznacza układ symboli na schemacie, w tym wstępne położenie nowo dodanych elementów. Komunikuje się z kontrolerem przez wspólny interfejs, co pozwala rozwijać edytor i algorytm niezależnie. W razie potrzeby korzysta z zewnętrznych bibliotek układu grafów. Komunikację z nimi obsługuje adapter, który przygotowuje dane w wymaganym formacie i przekształca wyniki. Dzięki temu zmiana wykorzystywanej biblioteki nie wpływa na pozostałe elementy systemu.

Taki podział oddziela strukturę wzoru od jego reprezentacji graficznej: model opisuje, *co* zostało zaprojektowane, moduł algorytmiczny określa, *jak* przedstawić to na schemacie, a edytor pośredniczy między użytkownikiem a tymi dwoma modułami.

1.2 Odpowiedzialności modułów

Moduł	**Główne odpowiedzialności**
UI	Renderowanie symboli SVG, obsługa gestów (klik, drag, zoom, pan), paleta symboli, panele właściwości, komunikaty błędów, wskaźniki postępu layoutu.
Kontroler	Koordynacja przepływu; przechowywanie bieżącego stanu aplikacji i historii operacji (undo/redo); wywoływanie operacji na modelu oraz żądań obliczenia układu; izolacja UI od logiki dziedzinowej.
Model wzoru	Jedne źródło prawdy o strukturze wzoru (graf oczek); operacje atomowe (dodanie/usunięcie/zmiana typu oczka, nowa warstwa); reguły walidacji poprawności wzoru; brak zależności od UI i algorytmu.
Algorytmy rozmieszczania	Wyznaczanie pozycji, orientacji i skal symboli; wstępne położenie nowo dodanych elementów; metryki jakości układu (nakładanie, proporcje).
Adapter bibliotek	Przygotowanie danych wejściowych dla zewnętrznej biblioteki układu grafów; konwersja wyników z powrotem do wspólnego formatu LayoutResult; izolacja reszty systemu od konkretnej implementacji.



Rysunek 1: Logiczna architektura systemu – główne moduły i zależności

2 Projekt modułów

2.1 Warstwy i zależności

Architektura jest warstwowa z wyraźnym kierunkiem zależności (warstwy wyższe zależą od niższych, nigdy odwrotnie):

1. **Presentation (UI)** – komponenty React, hooki; zależy wyłącznie od Kontrolera.
2. **Application (Kontroler)** – stan, historia, koordynacja; zależy od Modelu wzoru i interfejsu Algorytmów rozmieszczania.
3. **Domain (Model wzoru)** – czysty TypeScript, bez Reacta i DOM; niezależny.
4. **Infrastructure** – Web Worker, adapter bibliotek, localStorage, File API, canvas; implementuje interfejsy zdefiniowane wyżej.

Kluczowe interfejsy (szkic TypeScript):

```
// Domain
interface Stitch {
  id: string;
  type: StitchType;          // sc | hdc | dc | tr | ch | slst | mr ...
  parentIds: string[];       // oczka, w które wbite
  round: number;
  // ... dodatkowe atrybuty (np. yarn-over count)
```

```
}

interface PatternGraph {
  stitches: Map<string, Stitch>;
  yarnPosition: string | null;   // id ostatniego oczka
  currentRound: number;
}

interface DomainAPI {
  createNewPattern(start: StartElement): PatternGraph;
  addStitch(graph: PatternGraph, type: StitchType, parentId: string): PatternGraph;
  removeLast(graph: PatternGraph): PatternGraph;
  // ... pozostałe operacje z US-01..US-05
  validateInsertion(type: StitchType, target: Stitch): boolean;
}

// Layout
interface LayoutNode {
  id: string;
  x: number; y: number;
  rotation: number;          // radiany, baza w stronę rodzica
  scaleX: number; scaleY: number;
  width: number; height: number;  // po skalowaniu
}

interface LayoutResult {
  nodes: LayoutNode[];
  seed: number;
  metrics: { overlapAreaRatio: number; aspectViolations: number };
}

interface LayoutService {
  compute(graph: PatternGraph, options?: LayoutOptions): Promise<LayoutResult>;
}
```

2.2 Moduły szczegółowe (katalogi)

- **model**/ – implementacja PatternGraph, operacje immutable, walidatory reguł szydełkowych.
- **layout**/ – algorytmy rozmieszczania + adapter do zewnętrznej biblioteki + worker entrypoint.
- **controller**/ (lub store/) – stan aplikacji, historia undo/redo, selektory, koordynacja wywołań.
- **ui/editor**/ – Canvas SVG, SymbolRenderer, InteractionLayer (hit-testing, selection).
- **ui/panels**/ – Palette, Properties, Toolbar, LayerIndicator.

- **persist/** – JSON schema + serializers; AutosaveService; FileExporter (SVG/PNG/JPEG/PDF).
- **symbols/** – definicje ścieżek SVG i metadanych (preferowane aspect ratio, anchor points) dla każdego typu oczka.

Zależności zewnętrzne są izolowane w warstwie infrastructure (adapter + Web Worker), co ułatwia testowanie domeny i layoutu bez przeglądarki (NFR-M1) oraz wymianę biblioteki układu grafów bez zmian w reszcie systemu.

3 Komunikacja

Aplikacja jest w pełni client-side, więc nie występują protokoły sieciowe ani konfiguracja serwerów. Komunikacja wewnętrzna:

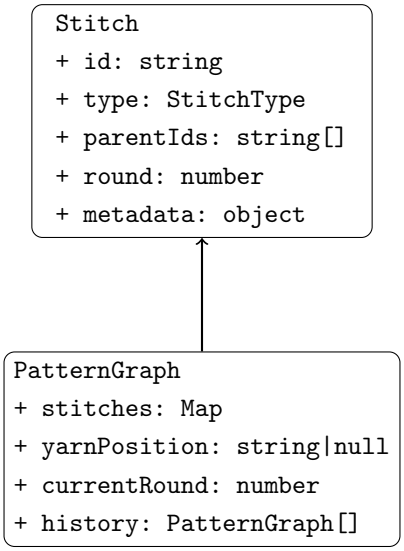
- **UI ↔ Kontroler** – React hooks + subskrypcje; zmiany stanu wywołują re-render tylko zainteresowanych komponentów.
- **Kontroler ↔ Model wzoru** – synchroniczne wywołania funkcji czystych (immutable updates). Każda operacja użytkownika tworzy nowy snapshot grafu i wpis w historii.
- **Kontroler ↔ Moduł algorytmiczny** – asynchroniczne: po każdej zmianie grafu wysyłany jest komunikat `postMessage` do Web Workera z serializowanym grafem + seedem. Worker zwraca LayoutResult. W czasie obliczeń UI pozostaje responsywny (NFR-P3).
- **Adapter ↔ biblioteka zewnętrzna** – wywołania API wybranej biblioteki (d3-force / WebCola lub innej); konwersja formatów po obu stronach.
- **Persist** – File System Access API (tam gdzie dostępne) lub klasyczne `<input type="file">` + download link; localStorage dla autosave co $\leq$ 30 s (NFR-R3).

Biblioteki komunikacyjne: natywne Web Worker API, structured clone algorithm (dla postMessage). Brak WebSocketów, HTTP clientów czy message brokerów.

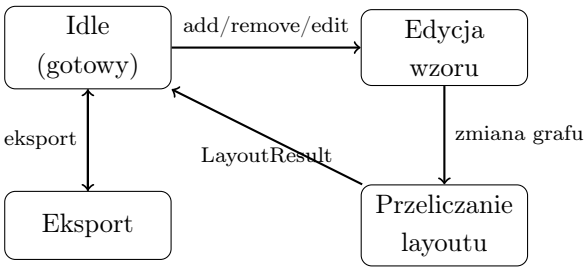
4 Główne komponenty – opisy i diagramy

4.1 Model danych (Domain)

Wierzchołki grafu odpowiadają oczkom. Krawędzie skierowane reprezentują relację „wbite w". Dodatkowo utrzymywana jest lista kolejności dodawania (dla undo i pozycji nici) oraz numer warstwy.



4.2 Stany aplikacji (uproszczony diagram stanów)



4.3 Aktywność: dodanie oczka (US-02)

1. Użytkownik wybiera typ z palety → stan „pending stitch type".
2. Kliknięcie symbolu na schemacie → walidacja reguły wbicia (Model wzoru).
3. Jeśli OK: nowy Stitch + aktualizacja grafu + wpis do historii (Kontroler).
4. Wywołanie LayoutService.compute() w Workerze (przez Adapter).
5. Po otrzymaniu wyniku: aktualizacja pozycji w Kontrolerze → re-render SVG.
6. Jeśli błąd walidacji: komunikat, brak zmiany modelu.

4.4 Zdarzenia kluczowe

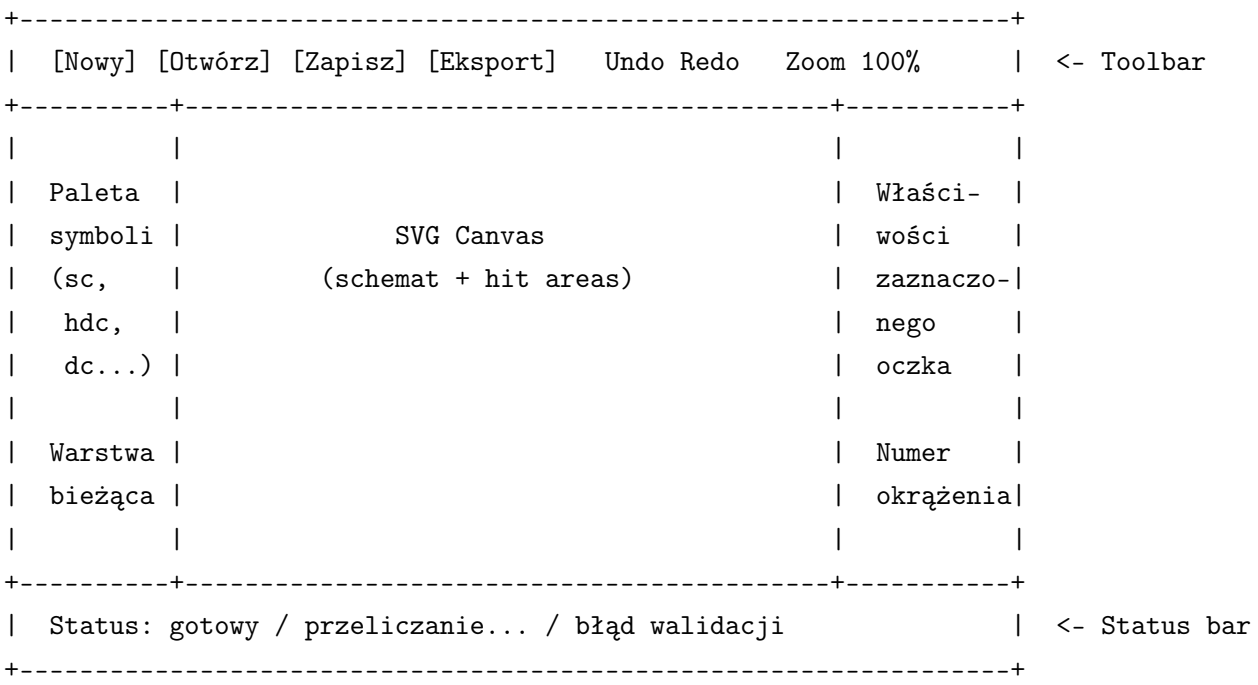
- `pattern:changed` – po każdej mutacji grafu.
- `layout:started / layout:finished / layout:failed`.

- `selection:changed`, `view:transformed` (zoom/pan).
- `history:undo / history:redo`.
- `persist:autosaved`, `export:requested`.

5 Wizja interfejsu użytkownika

Interfejs jest jednookienkowy, zorientowany na maksymalną przestrzeń roboczą dla schematu.

5.1 Główne obszary (wireframe tekstowy)



5.2 Kluczowe interakcje

- Kliknięcie symbolu w palecie + kliknięcie na schemacie = dodanie oczka (US-02).
- Kółko myszy / przyciski +/− = zoom względem kursora (US-07).
- Przeciąganie tła = pan.
- Ctrl+Z / Ctrl+Y = undo/redo (US-06).
- Podwójne kliknięcie lub panel właściwości = zmiana typu / parametrów.
- Wskaźnik postępu / spinner na canvasie podczas długiego layoutu (>200 ms).

Wizualizacje (makiety) zostaną doprecyzowane w kolejnej iteracji dokumentu lub w osobnym załączniku (Figma / szkice). Na obecnym etapie wystarcza opis obszarów i mapowanie na historyjki użytkownika.

6 Zewnętrzne interfejsy

6.1 Format pliku projektu (JSON)

Własny, wersjonowany format JSON (schema w osobnym pliku JSON Schema):

```
{
  "version": "1.0",
  "meta": { "title": "...", "created": "...", "seed": 12345 },
  "pattern": {
    "stitches": [ { "id": "...", "type": "dc", "parents": ["..."], "round": 2 }, ... ],
    "yarnPosition": "...",
    "currentRound": 3
  },
  "view": { "zoom": 1.0, "panX": 0, "panY": 0 }   // opcjonalnie
}
```

Zachowanie 100% danych strukturalnych (NFR-R1). Niepoprawny plik → komunikat błędu, brak awarii (NFR-R2).

6.2 Eksport graficzny

- **SVG** – natywny, wektorowy, zachowuje symbole jako `<g>` / `<path>`.
- **PNG / JPEG** – rasteryzacja SVG przez OffscreenCanvas lub tymczasowy `<canvas>`.
- **PDF** – jsPDF + svg2pdf.js lub podobna biblioteka; jedna strona A4 / Letter z dopasowanym schematem.

Brak integracji z zewnętrznymi usługami (brak OAuth, API chmurowych itp.). Standardy: SVG 1.1 / 2, JSON (RFC 8259), File API / File System Access API.

7 Stos technologiczny

Poniższa tabela przedstawia technologie przyjęte w projekcie wraz z rozważanymi alternatywami i uzasadnieniem wyboru. Aplikacja działa w całości w przeglądarce, dlatego narzędzia takie jak Node.js czy Vite są wykorzystywane wyłącznie na etapie tworzenia, testowania i budowania aplikacji.

Moduł / Komponent	**Wybrana technologia**	**Rozważane alternatywy**	**Uzasadnienie wyboru**
Platforma	aplikacja webowa	Aplikacja desktopowa: WPF/C#, Avalonia/C#, Electron	Dostęp bez instalacji, jedna implementacja dla różnych systemów operacyjnych.
Framework	React	Vue	Dojrzały ekosystem, model komponentowy, dobra współpraca z niemutowalnym stanem.
Język	TypeScript	JavaScript	Statyczne typowanie złożonego modelu danych, bezpieczniejsza refaktoryzacja, wspólne kontrakty między modułami.
Renderowanie schematu	SVG	Canvas (Konva / react-konva), czysty HTML Canvas	Grafika wektorowa, każdy symbol jako osobny element, naturalny format eksportu; Canvas wymagałby osobnego eksportu do SVG i własnej obsługi zaznaczania.
Narzędzie budowania	Vite	Webpack	Szybki serwer deweloperski, gotowa obsługa TypeScriptu i Reacta.
Środowisko uruchomieniowe (dev)	Node.js + npm	–	npm dostępny razem z Node.js bez dodatkowej instalacji; wersje przypięte w `.nvmrc` i lockfile'u zapewniają powtarzalność środowiska.
Repozytorium i CI/CD	GitHub + GitHub Actions	GitLab CI	Code review przez pull requesty, automatyczne testy przy każdej zmianie, wcześniejsze doświadczenie z GitHub Actions.
Zarządzanie stanem	Zustand (lub Jotai)	Redux Toolkit, React Context	Lekki store, łatwe middleware undo/redo, mniejszy boilerplate.
Silnik layoutu	d3-force / WebCola	Własna implementacja, cytoscape.js	Dojrzałe, dobrze udokumentowane implementacje force-directed + constraint-based; możliwość uruchomienia w Web Workerze; izolacja przez adapter.

Moduł / Komponent	**Wybrana technologia**	**Rozważane alternatywy**	**Uzasadnienie wyboru**
Testy	Vitest + Testing Library	Jest	Natywna integracja z Vite, szybkie uruchamianie, dobre wsparcie TypeScript.
Eksport PDF	jsPDF + svg2pdf	pdfkit, puppeteer	Działa w pełni po stronie klienta, bez potrzeby serwera.

7.1 Uzasadnienie wyboru podejścia do layoutu

Na podstawie literatury (m.in. Greer & Mould 2025 – *Modeling Crochet Patterns with a Force-directed Graph Layout*) oraz analizy istniejących narzędzi (CrochetPARADE, Stitch Fiddle) wybrano podejście force-directed / constraint-based. Nie tworzymy nowego algorytmu od zera; wartość inżynierska leży w integracji, dostosowaniu sił (orientacja symboli względem rodzica, preferowane proporcje H/W, minimalizacja nakładania) oraz zapewnieniu interaktywności (Web Worker, determinizm przy seedzie). Adapter bibliotek pozwala na ewentualną wymianę implementacji bez wpływu na resztę systemu.

7.2 Środowisko deweloperskie

Node.js + npm/pnpm, ESLint + Prettier, Git. Testy w CI (GitHub Actions). Brak zależności od konkretnego systemu operacyjnego – rozwój możliwy na Windows / macOS / Linux. Wersje narzędzi są przypięte (`.nvmrc`, `package-lock.json` / `pnpm-lock.yaml`).